UPTD SMP NEGERI 2 TANGERANG SELATAN

Mata Pelajaran: Informatika • Kelas IX

BUKU MATERI

Berpikir Komputasional dalam Algoritma dan Pemrograman

BAB 2 • Tugas 1

Modularisasi • Library • Flowchart • Pseudocode
Pemrograman Visual (Scratch/Blockly) • Transisi ke Pemrograman Tekstual

Pembuat Soal / Materi	Waryanto, ST
Satuan Pendidikan	UPTD SMPN 2 Tangsel
Kelas	IX (9.1 – 9.10)
Materi Pokok	Berpikir Komputasional

Buku ini disusun sebagai materi pendukung Tugas 1 BAB 2.
Isi materi disarikan dari konsep-konsep yang diujikan dalam soal kuis interaktif.

DAFTAR ISI

1. Pengantar Berpikir Komputasional 3

2. Modularisasi dalam Pemrograman 4

3. Library (Pustaka) dalam Pemrograman 6

4. Flowchart (Diagram Alir) 8

5. Pseudocode 11

6. Pemrograman Visual: Scratch & Blockly 13

7. Transisi dari Pemrograman Visual ke Tekstual 15

8. Dekomposisi Masalah 16

9. Evaluasi Efektivitas Solusi 17

10. Ringkasan & Kunci Konsep 18

11. Latihan Pemahaman 19

12. Glosarium 21

1 Pengantar Berpikir Komputasional

Berpikir komputasional (*computational thinking*) adalah cara berpikir untuk memecahkan masalah dengan pendekatan yang dapat dijalankan oleh komputer. Dalam konteks algoritma dan pemrograman, berpikir komputasional membantu kita merancang solusi yang sistematis, efisien, dan dapat digunakan ulang.

Empat pilar utama berpikir komputasional:

Pilar	Penjelasan
Dekomposisi	Memecah masalah besar menjadi bagian-bagian kecil yang lebih mudah dikelola.
Pengenalan Pola	Menemukan kesamaan atau pola yang berulang dalam masalah.
Abstraksi	Menyaring informasi penting dan mengabaikan detail yang tidak relevan.
Algoritma	Menyusun langkah-langkah penyelesaian masalah secara berurutan dan jelas.

■ Ingat

Dalam materi BAB 2 ini, fokus utama adalah **dekomposisi** (modularisasi) dan cara merepresentasikan algoritma melalui **flowchart** serta **pseudocode**, baik di pemrograman visual maupun tekstual.

Materi ini relevan dengan kemampuan yang diujikan: memahami modularisasi, library, flowchart, pseudocode, blok custom di Scratch/Blockly, serta cara beralih dari pemrograman visual ke tekstual.

2 Modularisasi dalam Pemrograman

Modularisasi adalah proses memecah program besar menjadi bagian-bagian kecil yang mandiri (disebut modul, prosedur, atau fungsi). Setiap modul mengerjakan satu tugas spesifik dan dapat digabungkan kembali untuk membentuk program utuh.

2.1 Mengapa Modularisasi Penting?

Bayangkan menulis program kasir yang berisi ratusan baris kode dalam satu blok panjang. Jika ada kesalahan di bagian perhitungan diskon, Anda harus mencari di seluruh program. Dengan modularisasi, Anda cukup memeriksa modul *hitungDiskon*.

Manfaat	Penjelasan
Keterbacaan (Readability)	Kode lebih mudah dibaca dan dipahami karena terstruktur.
Kemudahan Pengujian	Setiap modul dapat diuji secara terpisah (unit testing).
Isolasi Error	Kesalahan (bug) dapat dilokalisasi pada modul tertentu.
Reusability	Modul dapat dipanggil berulang kali dengan data berbeda.
Kolaborasi	Anggota tim dapat mengerjakan modul berbeda secara paralel.

2.2 Contoh Modularisasi

Tanpa modularisasi (semua logika digabung di program utama):

```
PROGRAM utama
  baca a, b, c
  total ← a + b + c
  rata ← total / 3
  tulis rata
AKHIR PROGRAM
```

Dengan modularisasi (logika dipisah ke prosedur/fungsi):

```
PROSEDUR hitungRata(nilai1, nilai2, nilai3)
  total ← nilai1 + nilai2 + nilai3
  rata ← total / 3
  KEMBALIKAN rata
AKHIR PROSEDUR

PROGRAM utama
  baca a, b, c
  hasil ← hitungRata(a, b, c)
  tulis hasil
AKHIR PROGRAM
```

■ Penting

Keunggulan pendekatan modular: prosedur **hitungRata** dapat dipanggil berulang kali untuk data berbeda tanpa menulis ulang logika perhitungan. Ini adalah inti dari reusability.

2.3 Parameter dan Return Value

Agar modul bersifat **generik** (bisa dipakai untuk data berbeda), modul harus menerima **parameter** sebagai masukan. Biasanya modul juga mengembalikan sebuah **nilai (return value)**.

- **Parameter** = data yang dikirim ke modul saat dipanggil (contoh: jari-jari r).
- **Return value** = hasil yang dikembalikan modul ke pemanggil (contoh: nilai luas).

■ Ingat

Pada flowchart, simbol input (jajaran genjang) diganti menjadi penerimaan parameter ketika flowchart diubah menjadi modul library yang bisa dipanggil berulang.

3 Library (Pustaka) dalam Pemrograman

Library adalah kumpulan fungsi atau prosedur siap pakai yang dapat digunakan ulang di berbagai program. Dengan library, programmer tidak perlu menulis ulang logika yang sudah ada, sehingga meningkatkan efisiensi pengembangan.

3.1 Hubungan Modularisasi dan Library

Modularisasi menghasilkan modul-modul kecil. Kumpulan modul yang berguna dan dapat dibagikan itulah yang disebut **library**. Jadi, library adalah hasil dari praktik modularisasi yang baik.

3.2 Keuntungan Menggunakan Library

Keuntungan	Penjelasan
Mengurangi duplikasi kode	Logika yang sama tidak perlu ditulis berkali-kali.
Mempercepat pengembangan	Cukup panggil fungsi yang sudah tersedia.
Memudahkan kolaborasi	Modul dapat dibagikan antar anggota tim / proyek.
Konsistensi	Satu sumber kebenaran; jika rumus berubah, cukup ubah di library.

■ Penting

Library **tidak** membuat program menjadi tidak dapat diubah, dan testing tetap diperlukan. Library juga tidak menghilangkan kebutuhan memahami logika dasar.

3.3 Perbandingan Dua Pendekatan

Pendekatan A: Menulis ulang kode perhitungan luas setiap kali dibutuhkan di berbagai bagian program.
Pendekatan B: Membuat satu fungsi *hitungLuas()* lalu memanggilnya di mana saja.

Aspek	Pendekatan A	Pendekatan B
Menerapkan modularisasi & library	Tidak	Ya
Rawan inkonsistensi jika rumus diubah	Ya (harus ubah di banyak tempat)	Tidak (ubah sekali)
Mendukung reusability	Tidak	Ya
Kemudahan debugging	Sulit (logika tersebar)	Mudah (logika terpusat)

3.4 Library di Pemrograman Visual

Di Scratch, fitur **My Blocks** (blok custom) adalah bentuk library pribadi. Siswa dapat membuat blok sendiri dengan parameter input dan bahkan mengembalikan nilai (reporter block). Blok tersebut dapat digunakan ulang di berbagai bagian program, bahkan lintas sprite jika di-share.

```
[ hitungLuasPersegiPanjang ]  ← blok custom (My Blocks)
  parameter: panjang, lebar
  return: panjang × lebar

Di script utama:
  set volume to (hitungLuasPersegiPanjang 10 5) × tinggi
```

■ Contoh

My Blocks mendukung parameter (berbagai tipe data) dan return value. Library bawaan Scratch (Motion, Looks, Sound, dll.) tetap dapat digunakan bersama My Blocks. Ini adalah implementasi nyata konsep modularisasi di pemrograman visual.

4 Flowchart (Diagram Alir)

Flowchart adalah representasi visual dari algoritma menggunakan simbol-simbol standar. Flowchart membantu memvisualisasikan alur logika secara jelas sebelum menulis kode program.

4.1 Simbol-Simbol Standar Flowchart

Simbol	Nama	Fungsi
Oval (Terminator)	Mulai / Selesai	Menandai awal dan akhir algoritma.
Persegi Panjang (Proses)	Operasi / Perhitungan	Menjalankan perhitungan atau perintah.
Belah Ketupat (Keputusan)	Percabangan kondisi	Menguji kondisi (Ya/Tidak, if-else).
Jajaran Genjang (I/O)	Input / Output	Membaca data (baca) atau menampilkan hasil (tulis).
Anak Panah (Flowline)	Alur eksekusi	Menunjukkan urutan langkah.

■ Ingat

Simbol oval muncul minimal **dua kali** (mulai & selesai). Tidak semua simbol wajib digunakan dalam setiap flowchart — hanya yang dibutuhkan oleh logika masalah.

4.2 Manfaat Flowchart

- Memvisualisasikan alur logika secara jelas dan mudah dipahami.
- Memudahkan deteksi kesalahan alur **sebelum** coding (menghemat waktu).
- Membantu komunikasi ide antar anggota tim.
- Berguna untuk program sederhana maupun kompleks.

Catatan: Flowchart **tidak menggantikan** testing program. Testing tetap wajib dilakukan.

4.3 Contoh Flowchart: Menghitung Luas Lingkaran

Alur sederhana (sekuensial):

```
[Mulai]
    |
  Input r          ← jajaran genjang
    |
  luas = 3.14 × r × r   ← persegi panjang (proses)
    |
  Output luas      ← jajaran genjang
    |
  [Selesai]
```

Jika flowchart ini akan diubah menjadi **modul library** yang bisa dipanggil berulang, simbol “Input r” diganti menjadi penerimaan **parameter**. Modul menjadi generik dan reusable.

4.4 Contoh Flowchart dengan Percabangan

■ Penting

Setiap simbol keputusan (belah ketupat) **wajib** memiliki cabang yang jelas (minimal Ya dan Tidak). Jika masalah membutuhkan pengecekan kondisi berulang tetapi flowchart hanya sekuensial tanpa keputusan/perulangan, maka flowchart tersebut **kurang efektif**.

5 Pseudocode

Pseudocode adalah representasi semi-formal dari algoritma yang menggunakan bahasa mirip manusia (bisa Bahasa Indonesia atau Inggris). Pseudocode membantu programmer merancang logika sebelum menulis kode dalam bahasa pemrograman tertentu.

5.1 Ciri-Ciri Pseudocode yang Baik

- Langkah-langkahnya **jelas, berurutan, dan tidak ambigu**.
- Mudah diterjemahkan ke bahasa pemrograman (Python, Java, dll.).
- Logika solusi **sesuai** dengan permasalahan yang ingin diselesaikan.
- Menggunakan penamaan deskriptif (contoh: *hitungRata*, bukan *x1*).

■ Tips

Panjang pseudocode **bukan** ukuran efektivitas. Pseudocode pendek yang jelas lebih baik daripada yang panjang tetapi membingungkan. Bahasa boleh Indonesia atau Inggris.

5.2 Contoh Pseudocode: Fungsi Maksimum

```
FUNGSI maksimum(a, b)
  JIKA a > b MAKA
    KEMBALIKAN a
  ELSE
    KEMBALIKAN b
  AKHIR JIKA
AKHIR FUNGSI
```

Analisis: Fungsi ini merupakan contoh modularisasi. Ia menerima dua parameter, mengembalikan satu nilai, dan logikanya dapat diuji secara terpisah. Fungsi dapat dipanggil berkali-kali dan mudah dipetakan ke blok custom Scratch.

5.3 Pseudocode sebagai Jembatan

Pseudocode berperan sebagai jembatan antara:
• Ide / masalah → solusi terstruktur
• Pemrograman visual (Scratch/Blockly) → pemrograman tekstual (Python, dll.)
• Anggota tim yang berbeda latar belakang teknis

6 Pemrograman Visual: Scratch & Blockly

Pemrograman visual menggunakan blok berbentuk puzzle yang disusun untuk membentuk program. Contoh populer: **Scratch** dan **Blockly**. Pendekatan ini sangat cocok untuk pemula karena mengurangi risiko syntax error.

6.1 Perbandingan Visual vs Tekstual

Aspek	Pemrograman Visual	Pemrograman Tekstual
Sintaks	Blok berbentuk puzzle	Teks dengan aturan ketat
Error	Sulit salah pasang blok	Mudah terjadi syntax error
Transisi	Mudah dipetakan ke pseudocode	Lebih dekat ke bahasa nyata
Struktur dasar	Variabel, percabangan, perulangan	Sama (variabel, if, loop)
Library	My Blocks / blok custom	import / #include / library bawaan

Keduanya mendukung konsep yang sama: variabel, percabangan, perulangan, dan modularisasi. Pemrograman visual lebih cocok sebagai **jembatan** menuju pemrograman tekstual. Scratch mampu merepresentasikan algoritma yang cukup kompleks (meski terbatas untuk skala sangat besar).

6.2 My Blocks sebagai Library

Di Scratch, fitur **My Blocks** memungkinkan siswa membuat blok sendiri dengan parameter. Fitur ini merupakan implementasi konsep **modularisasi** dalam pemrograman visual.

- My Blocks dapat memiliki parameter input (angka, teks, boolean).
- My Blocks dapat mengembalikan nilai (reporter block).
- Blok custom berfungsi sebagai library pribadi yang reusable.
- Library bawaan Scratch tetap dapat digunakan bersama My Blocks.

■ Contoh

Contoh nyata: siswa membuat blok custom “hitung volume” dengan parameter p, l, t, lalu menggunakannya di beberapa sprite berbeda. Blok tersebut berperan sebagai library yang mengenkapsulasi logika perhitungan.

7 Transisi dari Pemrograman Visual ke Tekstual

Siswa yang sudah mahir di Blockly/Scratch perlu beralih ke bahasa tekstual (misalnya Python). Transisi yang baik tidak dilakukan dengan menyalin buta, melainkan dengan memahami logika terlebih dahulu.

7.1 Langkah yang Disarankan

No	Langkah
1	Buat flowchart atau pseudocode dari program visual yang sudah ada.
2	Kenali kesamaan struktur: urutan, percabangan (if), perulangan (loop).
3	Pahami konsep variabel, parameter, dan return value.
4	Terjemahkan langkah demi langkah ke sintaks bahasa tekstual.
5	Perhatikan pesan error — jangan diabaikan; error adalah petunjuk belajar.

■ Penting

Langkah terbaik: buat flowchart/pseudocode dari program Blockly-nya, lalu terjemahkan langkah demi langkah ke sintaks tekstual. Jangan langsung menulis kode tanpa menganalisis logika, dan jangan menghapus semua blok lalu mulai dari nol tanpa memahami konsep yang sama.

Pseudocode dan flowchart adalah jembatan terbaik dari visual ke tekstual karena keduanya fokus pada **logika**, bukan sintaks.

8 Dekomposisi Masalah

Dekomposisi adalah bagian dari berpikir komputasional yang berarti memecah masalah/program menjadi bagian-bagian kecil yang manageable. Modularisasi adalah penerapan dekomposisi di dunia pemrograman.

8.1 Contoh Penerapan Dekomposisi

Program kasir dapat dipecah menjadi modul-modul berikut:

- Modul **input barang** — menerima data barang dan harga.
- Modul **hitung total** — menjumlahkan harga semua barang.
- Modul **hitung diskon** — menerapkan aturan diskon.
- Modul **cetak struk** — menampilkan hasil ke pengguna.
- Memisahkan logika tampilan (UI) dari logika bisnis (perhitungan).

Praktik yang **tidak** termasuk dekomposisi yang baik:
• Menulis seluruh logika dalam satu prosedur panjang tanpa pemisahan.
• Menggabungkan semua variabel menjadi satu variabel besar.

8.2 Praktik Terbaik Perancangan

- Mulai dari gambaran kasar (high-level), lalu detailkan modul demi modul (top-down).
- Pastikan setiap keputusan memiliki cabang yang jelas (Ya/Tidak).
- Uji logika flowchart/pseudocode dengan data contoh sebelum coding.
- Gunakan nama prosedur yang deskriptif agar mudah dipahami.
- Jangan abaikan kasus khusus (edge case) — justru penting untuk diantisipasi.

9 Evaluasi Efektivitas Solusi

Setelah merancang flowchart atau pseudocode, kita perlu mengevaluasi apakah solusi tersebut efektif.

9.1 Kriteria Evaluasi Pseudocode

- Apakah langkah-langkahnya jelas, berurutan, dan tidak ambigu?
- Apakah pseudocode dapat diterjemahkan ke bahasa pemrograman dengan mudah?
- Apakah logika solusi sesuai dengan permasalahan yang ingin diselesaikan?

9.2 Kriteria Evaluasi Flowchart

- Apakah flowchart merepresentasikan logika yang dibutuhkan masalah (termasuk percabangan/perulangan jika diperlukan)?
- Apakah setiap keputusan memiliki cabang lengkap?
- Apakah alur dari mulai sampai selesai logis dan tidak ada jalan buntu?

■ Penting

Jika masalah membutuhkan pengecekan kondisi berulang, tetapi flowchart hanya berisi 12 simbol proses berurutan tanpa keputusan atau perulangan, maka flowchart tersebut **kurang efektif** karena tidak merepresentasikan logika yang dibutuhkan. Jumlah simbol banyak bukan jaminan efektivitas.

10 Ringkasan & Kunci Konsep

Konsep	Definisi Singkat
Modularisasi	Memecah program besar menjadi bagian kecil yang mandiri.
Library	Kumpulan fungsi/prosedur siap pakai yang reusable.
Parameter	Data masukan agar modul bersifat generik.
Return value	Nilai yang dikembalikan modul ke pemanggil.
Reusability	Kemampuan kode digunakan ulang tanpa ditulis ulang.
Isolasi error	Bug dapat dilokalisasi pada modul tertentu (mudah debugging).
Flowchart	Diagram alir visual dengan simbol standar (oval, persegi, belah ketupat, jajaran genjang).
Pseudocode	Deskripsi algoritma semi-formal mirip bahasa manusia.
My Blocks	Fitur Scratch untuk membuat blok custom = bentuk modularisasi visual.
Dekomposisi	Memecah masalah menjadi bagian manageable (pilar berpikir komputasional).
Top-down design	Merancang dari gambaran kasar lalu detailkan modul demi modul.

Dengan membuat **pseudocode** dan **flowchart** terlebih dahulu, siswa dapat memahami dan merancang solusi secara sistematis serta efisien sebelum menulis kode program — baik di lingkungan visual maupun tekstual.

11 Latihan Pemahaman

Kerjakan soal-soal berikut untuk menguji pemahaman. Jawaban dapat didiskusikan bersama guru atau teman sekelas.

A. Pilihan Ganda

1. Jika siswa ingin memodularisasi proses perhitungan luas lingkaran agar dapat digunakan ulang di program lain, langkah manakah yang paling tepat?
a. Mengubah seluruh flowchart menjadi satu blok besar tanpa input/output
b. Membuat prosedur/fungsi terpisah bernama hitungLuas(r) yang menerima parameter jari-jari dan mengembalikan nilai luas
c. Menghapus simbol mulai dan selesai agar flowchart lebih singkat
d. Mengganti nilai 3.14 menjadi variabel π tanpa mengubah struktur

2. Manakah pernyataan yang paling tepat mengenai keunggulan prosedur hitungRata dibanding menulis semua perhitungan di program utama?
a. Pseudocode menjadi lebih panjang sehingga lebih sulit dipahami
b. Prosedur dapat dipanggil berulang kali untuk data berbeda tanpa menulis ulang logika (modularisasi & library)
c. Nilai a, b, c tidak perlu dideklarasikan karena sudah ada di prosedur
d. Operator pembagian (/) hanya boleh digunakan di dalam prosedur

3. Blok custom di Scratch dengan parameter dan return value berperan sebagai…
a. Variabel global yang menyimpan nilai tetap
b. Library/modul yang mengenkapsulasi logika agar dapat digunakan ulang
c. Struktur perulangan (loop)
d. Kondisi percabangan (if-else)

B. Benar / Salah (pilih semua yang benar)

4. Manakah pernyataan yang benar mengenai modularisasi?
■ Memecah program besar menjadi bagian-bagian kecil yang mandiri
■ Membuat kode lebih mudah dibaca, diuji, dan diperbaiki
■ Mengharuskan semua variabel menjadi global
■ Memungkinkan penggunaan ulang (reusability) kode
■ Hanya berlaku untuk pemrograman tekstual, tidak untuk Scratch

5. Keuntungan menggunakan library meliputi…
■ Mengurangi duplikasi kode
■ Mempercepat pengembangan program baru
■ Membuat program menjadi tidak dapat diubah sama sekali
■ Memudahkan kolaborasi karena modul dapat dibagikan
■ Menghilangkan kebutuhan testing

C. Isian Singkat

6. Proses memecah program besar menjadi bagian-bagian kecil yang mandiri disebut _____. Bagian-bagian tersebut dapat dikemas menjadi _____ agar dapat digunakan ulang di program lain.

7. Di Scratch, fitur _____ memungkinkan siswa membuat blok sendiri dengan parameter. Fitur ini merupakan implementasi konsep _____ dalam pemrograman visual.

8. Untuk berpindah dari Blockly ke bahasa tekstual, siswa sebaiknya menuliskan ulang logika dalam bentuk _____ atau _____, lalu menerjemahkannya ke sintaks bahasa target.

9. Salah satu manfaat utama modularisasi adalah _____ (kode dapat dipakai ulang) dan memudahkan _____ karena kesalahan dapat dilokalisasi pada modul tertentu.

10. Agar sebuah modul bersifat generik, modul tersebut harus menerima _____ sebagai masukan dan biasanya mengembalikan sebuah _____.

Kunci Jawaban Singkat

No	Jawaban
1	b
2	b
3	b
4	Pernyataan 1, 2, 4 benar
5	Pernyataan 1, 2, 4 benar
6	modularisasi ; library
7	My Blocks ; modularisasi
8	flowchart ; pseudocode
9	reusability ; debugging
10	parameter ; nilai/return

12 Glosarium

Istilah	Arti
Algoritma	Urutan langkah logis untuk menyelesaikan suatu masalah.
Debugging	Proses menemukan dan memperbaiki kesalahan (bug) dalam program.
Dekomposisi	Memecah masalah kompleks menjadi bagian-bagian lebih kecil.
Edge case	Kasus khusus / kondisi ekstrem yang jarang terjadi tetapi perlu diantisipasi.
Flowchart	Diagram alir yang merepresentasikan algoritma secara visual.
Fungsi / Prosedur	Blok kode mandiri yang mengerjakan tugas spesifik (bentuk modul).
Library	Kumpulan fungsi/prosedur siap pakai yang dapat digunakan ulang.
Modularisasi	Proses memecah program menjadi modul-modul mandiri.
My Blocks	Fitur Scratch untuk membuat blok kustom (custom block).
Parameter	Variabel masukan yang diterima oleh fungsi/prosedur.
Pseudocode	Deskripsi algoritma menggunakan bahasa semi-formal mirip manusia.
Reporter block	Blok Scratch yang mengembalikan nilai (return value).
Return value	Nilai yang dikembalikan oleh fungsi kepada pemanggilnya.
Reusability	Kemampuan suatu modul/kode untuk digunakan ulang.
Sprite	Objek/karakter di Scratch yang dapat menjalankan skrip.
Syntax error	Kesalahan penulisan kode yang melanggar aturan bahasa pemrograman.
Top-down design	Pendekatan merancang dari gambaran umum ke detail.

Selamat Belajar!

Semoga materi ini membantu memahami konsep berpikir komputasional, modularisasi, library, flowchart, dan pseudocode. Gunakan waktu sebaik-baiknya dan jaga kejujuran dalam mengerjakan tugas.

— Waryanto, ST • UPTD SMPN 2 Tangsel —